operations with EF Core via its *context*, an object that derives from `DbContext`, which is central to working with EF Core.

EF Core is a lot more powerful than just replacing the boilerplate code required to execute commands against a database. We will explore how it can be used to generate a database from your application model and then keep your database schema in sync with the model with a feature called *migrations*. We will also look at the conventions EF Core uses to map your model to the database and how you can use configuration to customize those mappings if needed.

This chapter will also cover a feature known as *scaffolding*. This feature combines your application model and database knowledge to quickly generate working CRUD pages for specific objects within the model. You'll understand the limitations of the code generated by the scaffolding tools and see how to improve them to comply with some of the software engineering principles we looked at in the last chapter.

By the end of this chapter, you will understand the role of EF Core and how to use it to execute commands against a relational database, manage that database's schema, and generate CRUD pages. EF Core is, however, a massive topic; we'll only scratch the surface of its capabilities and use in this book. To get the best out of this fantastic tool, you should obtain a copy of the excellent *Entity Framework Core in Action* (2nd ed.) by Jon P. Smith (http://mng.bz/vXr4) and refer to the official documentation (https://docs.microsoft.com/en-us/ef/core/).

8.1 What is Entity Framework Core?

EF Core is an object-relational mapping (ORM) tool. Its role is to map between objects (your application model) and the relational world (your